MOTHLINE
BACKUP + GITHUB RICE FAQ

A safe, repeatable command guide for preserving and publishing the CachyOS / Niri / Noctalia v5 rice

BACKUP	VERIFY	PUBLISH
Create a dated local archive before changing the rice.	Inspect the archive and Git changes before trusting them.	Commit only reviewed, private-safe files to GitHub.

Golden rule

Back up first. Review second. Push third. Never place passwords, API keys, tokens, browser profiles, SSH keys, or private personal files in the rice repository.

How to use this guide

Run one block at a time in a normal terminal. Read the output before continuing. Commands beginning with # are comments. Replace the repository path in the setup block if your local rice folder uses a different name.

Prepared for Spencer Kilgore // Updated 19 August 2026

1 // Safe setup

Q: What variables should I set before running the commands?

These task-specific variables keep paths readable without repurposing system variables. The backup is placed in Downloads by default; change only RICE_REPO if necessary.

```
set -l MOTHLINE_REPO "$HOME/mothline-rice"
set -l MOTHLINE_BACKUP_ROOT "$HOME/Downloads/Mothline-Backups"
set -l MOTHLINE_STAMP (date +%Y-%m-%d_%H%M)
set -l MOTHLINE_ARCHIVE "$MOTHLINE_BACKUP_ROOT/mothline-$MOTHLINE_STAMP.tar.zst"

mkdir -p "$MOTHLINE_BACKUP_ROOT"
test -d "$MOTHLINE_REPO"; and echo "Repository found"
echo "$MOTHLINE_ARCHIVE"
```

Q: Why quote every path?

Quoting prevents spaces or special characters in a filename from splitting one intended path into several arguments.

Q: Should these commands be run with sudo?

No. The personal configuration and repository commands should run as your normal user. Use sudo only for a separately identified system-level restore, never for the whole guide.

CAUTION: Do not create root-owned files inside your home configuration or Git repository.

Q: How do I confirm there is enough free space?

Check the filesystem containing Downloads before making the archive.

```
df -h "$HOME/Downloads"
du -sh "$HOME/.config" "$MOTHLINE_REPO" 2>/dev/null
```

2 // Create the Mothline backup

Q: What does the main backup include?

This captures the relevant user configurations and rice assets while excluding caches. Missing optional paths are tolerated by tar.

```
tar --zstd -cvf "$MOTHLINE_ARCHIVE" \
--ignore-failed-read \
--exclude='.cache' \
--exclude='Cache' \
-C "$HOME" \
.config/niri \
.config/noctalia \
.config/kitty \
.config/fastfetch \
.config/starship.toml \
.config/MangoHud \
.config/fish/functions \
.local/share/sddm/themes/mothline \
mothline-rice
```

Q: What about Firefox and Fluxer?

Browser and chat-client profiles may contain logins, cookies, message data, or tokens. Back up only the custom CSS/theme files you deliberately identified, preferably by copying clean versions into the private backup staging area or the sanitized rice repository.

CAUTION: Do not archive an entire browser or Fluxer profile into a public Git repository.

Q: How do I add a clean start page or Fluxer CSS file?

If the files are already stored inside the rice repository, the main archive includes them. Otherwise copy the known theme file into a staging folder first; use the exact source path you verified.

```
mkdir -p "$HOME/Downloads/Mothline-Staging/firefox"
mkdir -p "$HOME/Downloads/Mothline-Staging/fluxer"

# Example only - replace each source with the verified file:
cp --verbose /verified/path/start.html \
"$HOME/Downloads/Mothline-Staging/firefox/"
cp --verbose /verified/path/mothline-fluxer.css \
"$HOME/Downloads/Mothline-Staging/fluxer/"
```

3 // Verify and restore

Q: How do I verify the archive was created successfully?

Check that it exists, inspect its size, test decompression, and list the first entries.

```
ls -lh "$MOTHLINE_ARCHIVE"
zstd -t "$MOTHLINE_ARCHIVE"
tar --zstd -tf "$MOTHLINE_ARCHIVE" | head -n 40
```

Q: How do I generate a checksum?

A SHA-256 checksum detects corruption or accidental changes after copying the archive.

```
sha256sum "$MOTHLINE_ARCHIVE" | tee "$MOTHLINE_ARCHIVE.sha256"
sha256sum -c "$MOTHLINE_ARCHIVE.sha256"
```

Q: How do I copy it to Dropbox?

Copy the archive and checksum to the mounted/synchronized Dropbox folder after confirming its actual local path.

CAUTION: Confirm Dropbox has finished syncing before deleting any local copy.

```
set -l MOTHLINE_DROPBOX "$HOME/Dropbox/Mothline-Backups"
mkdir -p "$MOTHLINE_DROPBOX"
cp -iv "$MOTHLINE_ARCHIVE" "$MOTHLINE_ARCHIVE.sha256" "$MOTHLINE_DROPBOX/"
ls -lh "$MOTHLINE_DROPBOX"
```

Q: How do I inspect a backup without restoring it?

Extract into a brand-new temporary review directory, never directly over the live configuration.

```
set -l MOTHLINE_REVIEW (mktemp -d "$HOME/Downloads/mothline-review.XXXXXX")
tar --zstd -xvf "$MOTHLINE_ARCHIVE" -C "$MOTHLINE_REVIEW"
echo "$MOTHLINE_REVIEW"
find "$MOTHLINE_REVIEW" -maxdepth 3 -type f | sort | head -n 80
```

Q: What is the safest restore method?

Extract into a review folder, compare the desired files, then copy only the selected configuration back. Log out or stop the affected application before replacing its live configuration.

CAUTION: Never extract an unreviewed archive directly into $HOME.

```
diff -ru "$HOME/.config/kitty" \
"$MOTHLINE_REVIEW/.config/kitty" | less

# After reviewing the difference:
cp -aiv "$MOTHLINE_REVIEW/.config/kitty/." "$HOME/.config/kitty/"
```

4 // Review the GitHub rice

Q: How do I confirm I am in the correct repository?

Enter the repository, show its root, remote, branch, and current status before changing anything.

```
cd "$MOTHLINE_REPO"
git rev-parse --show-toplevel
git remote -v
git branch --show-current
git status --short --branch
```

Q: How do I see exactly what changed?

Review the summary, unstaged patch, and staged patch independently.

```
git diff --stat
git diff
git diff --cached
```

Q: How do I scan for obvious private material before staging?

This is a warning scan, not a guarantee. Review every match and every new file yourself.

CAUTION: Do not publish SSH keys, VPN credentials, email settings with secrets, personal paths containing private data, cookies, or API tokens.

```
rg -n -i \
'(password|passwd|api[_-]?key|secret|token|authorization|cookie|private[_-]?key)' \
. \
--glob '!*.png' --glob '!*.jpg' --glob '!*.webp' \
--glob '!.git/**' || true

git status --short
```

Q: What should .gitignore normally exclude?

At minimum, ignore secrets, archives, editor debris, logs, and temporary review output. Merge these rules with the existing file instead of overwriting it blindly.

```
# Suggested entries to review for .gitignore
*.tar
*.tar.gz
*.tar.zst
*.log
*.tmp
*.bak
*.swp
*secret*
*token*
.env
.env.*
private/
backups/
review/
```

5 // Stage, commit, and push

Q: What is the safest way to stage changes?

Stage named files or directories, then inspect the staged patch. Avoid git add . until the entire worktree has been reviewed.

```
git add README.md
git add fluxer/
git add sddm/
git add screenshots/

git status --short
git diff --cached --stat
git diff --cached
```

Q: How do I remove something from staging without deleting it?

Restore only the index copy. The local file remains in place.

```
git restore --staged path/to/file
git status --short
```

Q: How do I commit the update?

Use a short subject describing the finished checkpoint.

```
git commit -m "Add final Mothline SDDM and Fluxer themes"
git status --short --branch
```

Q: Should I pull before pushing?

Fetch first and inspect whether the local branch is behind. If the remote contains changes, use a rebase pull only after the worktree is clean and the backup exists.

CAUTION: Do not start a rebase with uncommitted work you have not backed up.

```
git fetch origin
git status --short --branch

# Run only if status says the branch is behind and the worktree is clean:
git pull --rebase

git push
```

Q: How do I verify the push?

Confirm the working tree is clean and show the latest commits and remote tracking status.

```
git status --short --branch
git log --oneline --decorate -n 5
git branch -vv
```

6 // Troubleshooting

Q: Why did a script report text not found for install.zsh?

The helper expected an exact text fragment that no longer matched the current install.zsh. Stop using the blind replacement, inspect the relevant section, and edit against the current file.

```
cd "$MOTHLINE_REPO"
rg -n 'sddm|fluxer|install|theme' install.zsh README.md
git diff -- install.zsh
git status --short
```

Q: Git says 'nothing to commit,' but I changed files.

Confirm the files are inside the repository, not ignored, and actually differ from the committed versions.

```
git rev-parse --show-toplevel
git status --short --untracked-files=all
git check-ignore -v path/to/file
git diff -- path/to/file
```

Q: Push was rejected because the remote contains work I do not have.

Fetch, inspect, and rebase only from a clean backed-up worktree. Resolve conflicts deliberately, test, then continue.

CAUTION: Never use a force push merely to bypass a rejection unless you have deliberately chosen to rewrite shared history.

```
git fetch origin
git status --short --branch
git log --oneline --left-right --graph HEAD...@{upstream}
git pull --rebase
# After resolving each conflict:
git add path/to/resolved-file
git rebase --continue
git push
```

Q: The backup command says a path does not exist.

Some Mothline components may live elsewhere or may not be installed. Locate the actual directory, update the list, and rerun with a new timestamp.

```
find "$HOME/.config" -maxdepth 2 \
\( -iname '*niri*' -o -iname '*noctalia*' -o -iname '*fastfetch*' \
-o -iname '*kitty*' -o -iname '*mangohud*' \) -print
find "$HOME/.local/share" -maxdepth 4 -iname '*mothline*' -print
```

7 // One-session checklist

STEP	ACTION	PASS CONDITION
1	Confirm paths	Repository, backup destination, and Dropbox destination are correct.
2	Create archive	Run the dated tar.zst backup before editing or pulling.
3	Verify archive	Test zstd, list contents, and create a SHA-256 checksum.
4	Review repository	Check branch, remote, status, unstaged diff, and untracked files.
5	Privacy scan	Inspect every match and ensure no credentials or private profiles are included.
6	Stage selectively	Add named files; inspect git diff --cached in full.
7	Commit	Use a clear checkpoint message and confirm the worktree state.
8	Synchronize	Fetch, rebase only if needed, then push.
9	Verify	Review git status, recent log, and remote tracking.
10	Retain backups	Keep at least one verified local copy and one verified off-device/synced copy.

Recommended next repository checkpoint

Add the finalized Fluxer CSS, SDDM theme, current screenshots, restore notes, and this FAQ. Keep diagnostic TXT outputs and backup archives outside the public repository unless a cleaned document is intentionally included.

MOTHLINE // preserve the work, inspect the change, publish with intent.